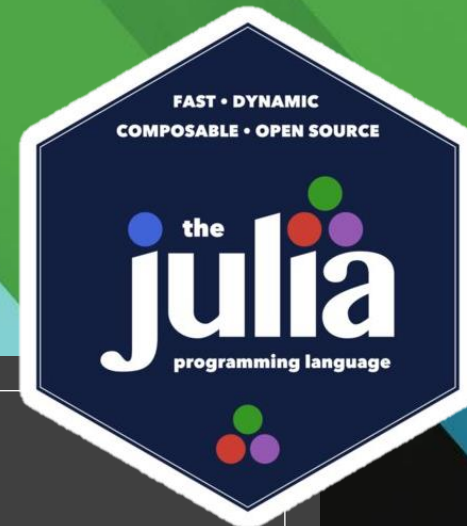


# InfiniteOpt



## INTRODUCTION TO JULIA, JUMP, AND INFINITEOPT

Prof. Joshua Pulsipher



# JUMP

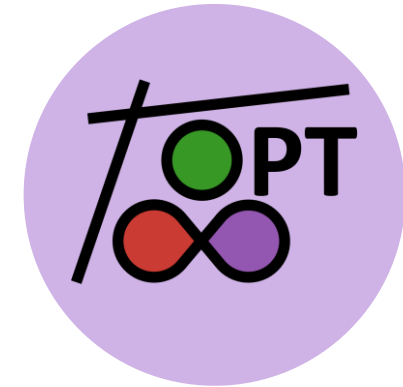
# TODAY'S TOPICS



CODING IN JULIA



MATHEMATICAL  
OPTIMIZATION VIA JUMP.JL



INFINITE-DIMENSIONAL  
OPTIMIZATION VIA  
INFINITEOPT.JL

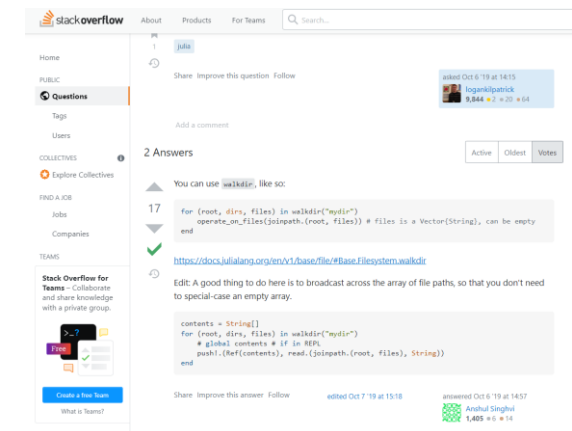
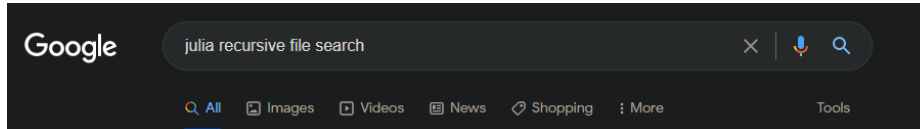


UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# LEARNING OUTCOMES

- Topic familiarity
  - Can briefly describe what Julia, JuMP.jl, and InfiniteOpt.jl are (and what they might be useful for)
  - Can “Google” the right things
- Programming expertise
  - Can setup basic Julia, JuMP.jl, and InfiniteOpt.jl workflows with appropriate reference materials
- Continued learning
  - Know what/where comes next in developing expertise with these tools
  - Identify resources to facilitate this learning



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

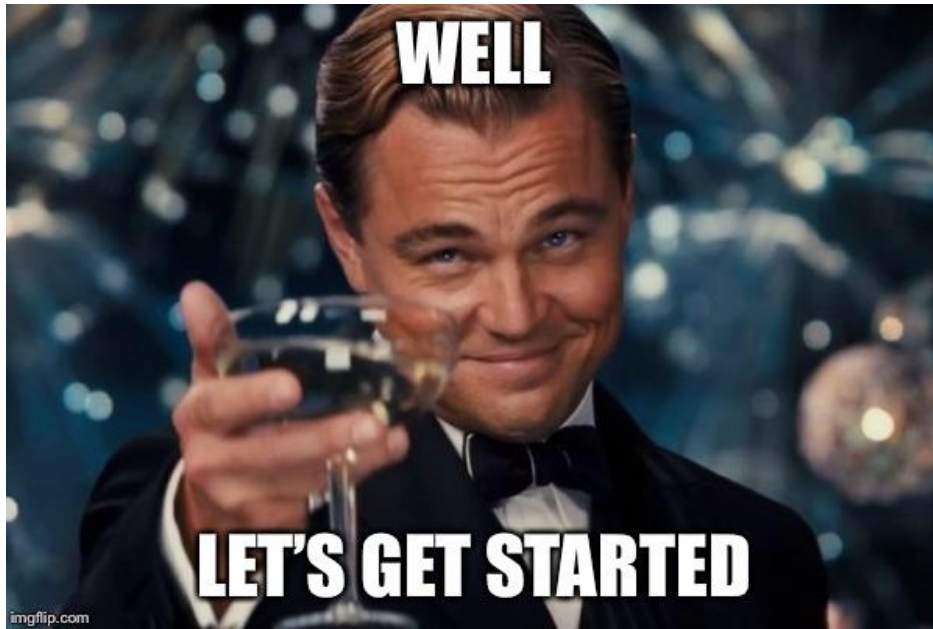
# COURSE SCHEDULE

- Introduction and installation (~30 minutes)
- Julia overview (~30 minutes)
- 15-minute Break
- JuMP overview (~60 minutes)
- 15-minute break
- InfiniteOpt overview (~90 minutes)

# QUICK SETUP START

- Please go to [https://github.com/infiniteopt/InfiniteOptTutorials/tree/main/short\\_course\\_4\\_hr](https://github.com/infiniteopt/InfiniteOptTutorials/tree/main/short_course_4_hr)
- Please open Part 1 in Colab
- Be sure to switch to the “Julia” runtime.





# Introduction

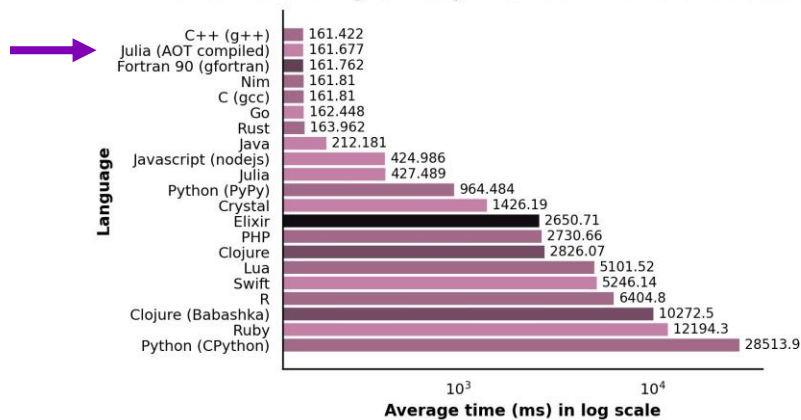
# WHY JULIA?



- **Fast** as C/C++/Fortran, but as **simple** as Python/MATLAB/R

## Speed comparison of various programming languages

Method: calculating  $\pi$  through the Leibniz formula 100000000 times



[github.com/coin-or/Ipopt.git](https://github.com/coin-or/Ipopt.git)

220k lines of code



[github.com/MadNLP/MadNLP.jl.git](https://github.com/MadNLP/MadNLP.jl.git)

20k lines of code

- **JuMP** provides intuitive and powerful platform for optimization



- **50+ solver library** + all AMPL and GAMS solvers
- Extensive amount of **constraint types** supported



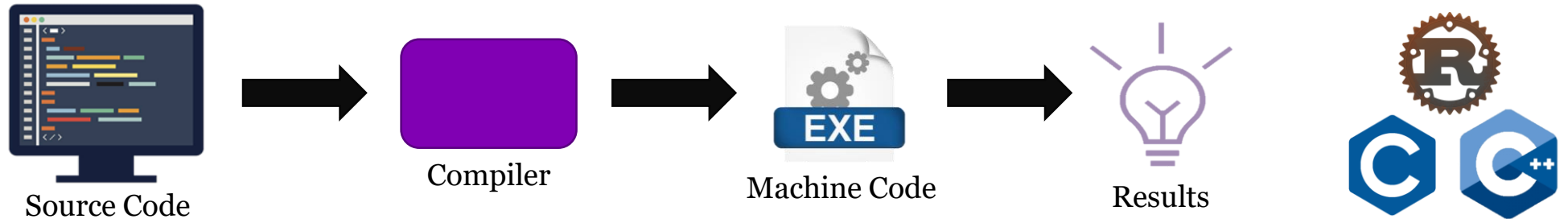
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

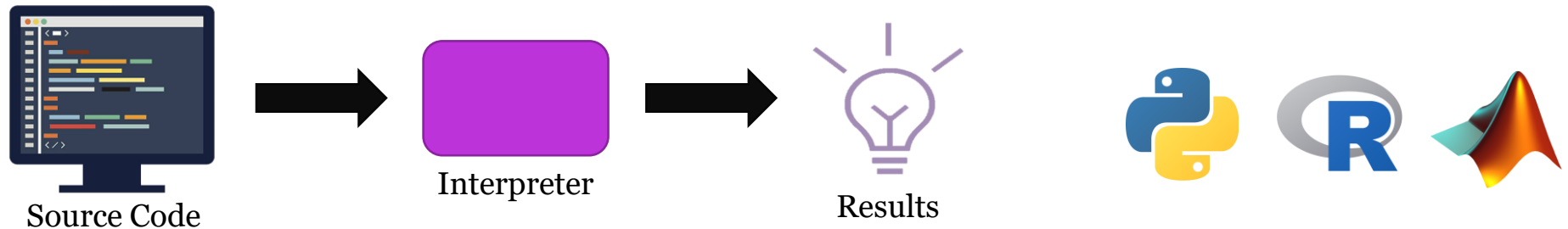


# PROGRAMMING LANGUAGE PARADIGMS

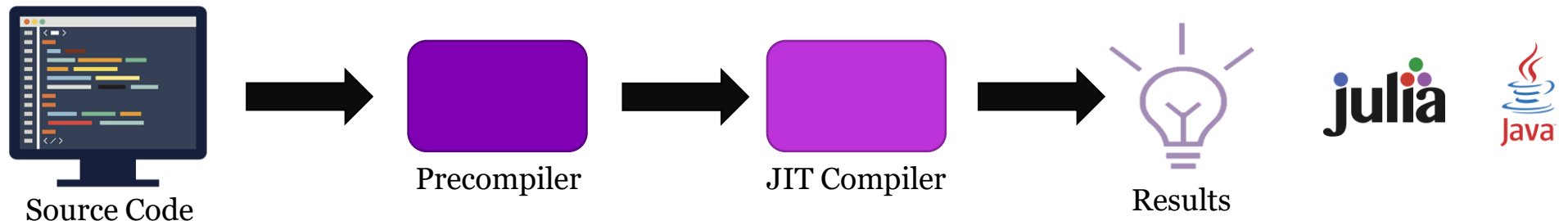
Compiled



Interpreted



Just-in-Time  
Compiled



\*Over simplified



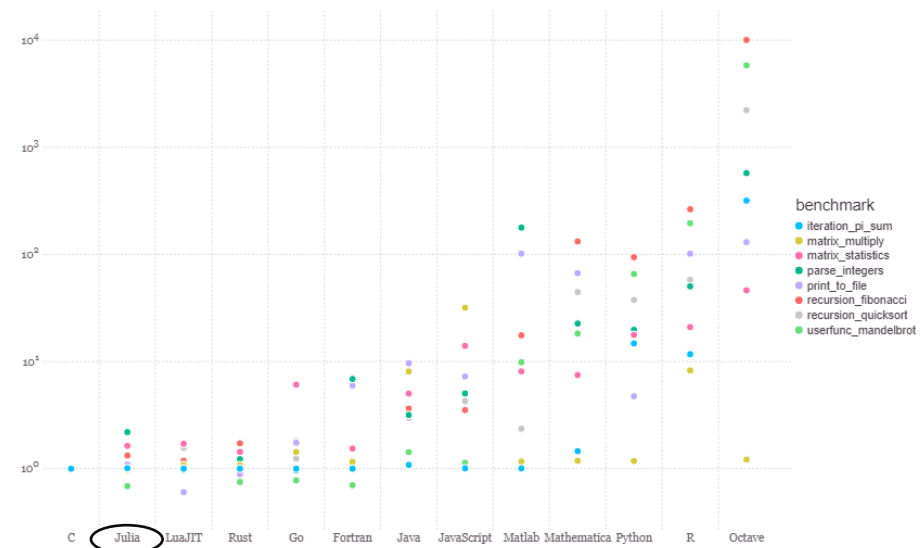
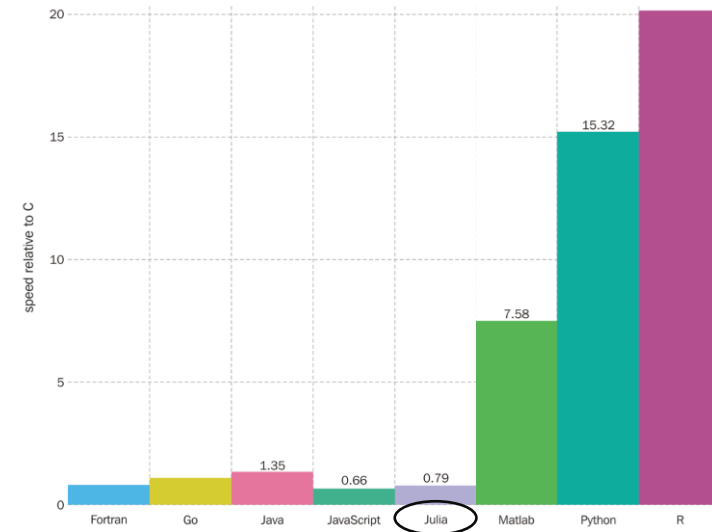
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING



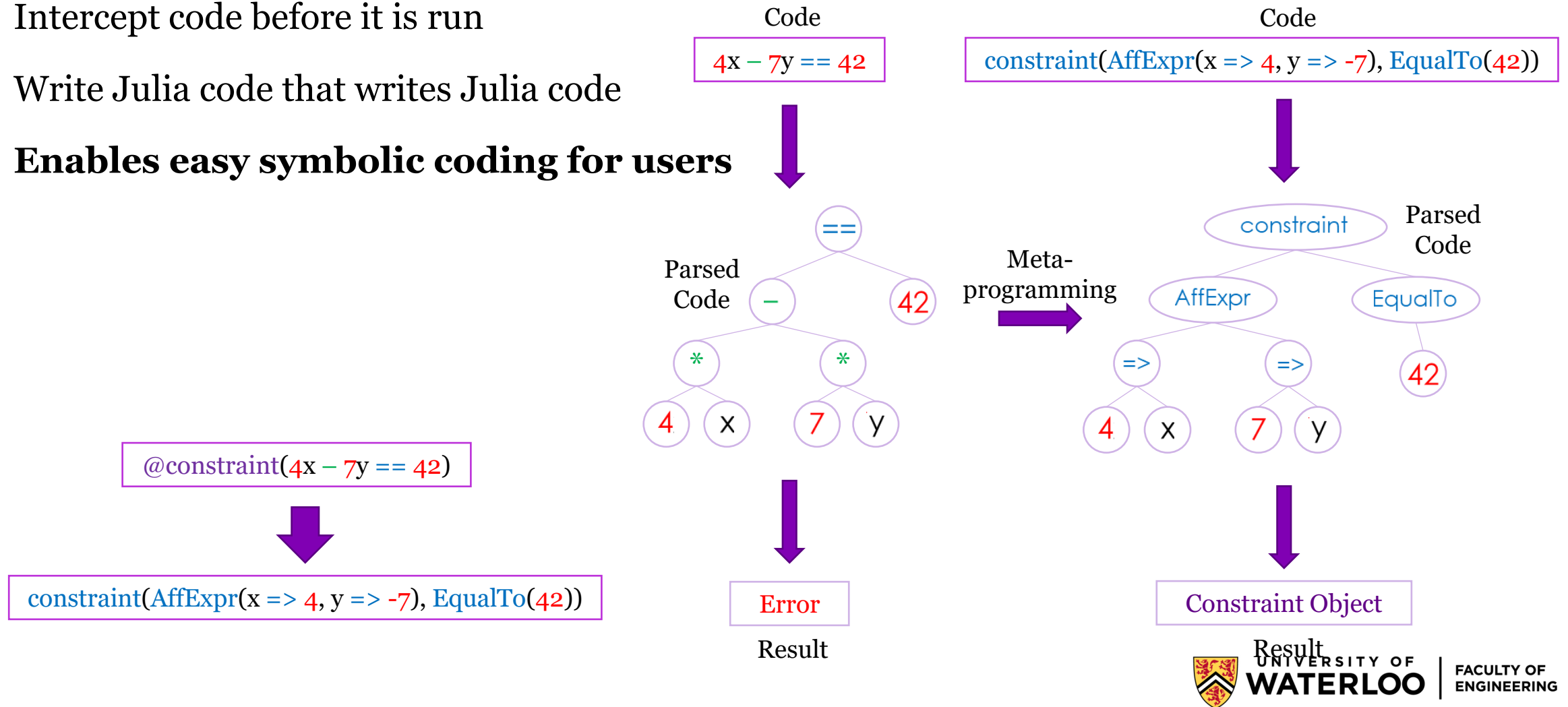
# PERFORMANCE

- Compiled Languages
  - Difficult “low-level” code
  - Cannot be modified while running
  - Not very portable between computers
  - Fast
- Interpreted Languages
  - Easy “high-level” syntax
  - Can be modified while running
  - Highly portable
  - Slow
- JIT Compiled Languages
  - Can be “high-level” (only Julia)
  - Highly portable
  - Can be modified while running
  - Fast\*



# METAPROGRAMMING (MACROS)

- Intercept code before it is run
- Write Julia code that writes Julia code
- Enables easy symbolic coding for users**

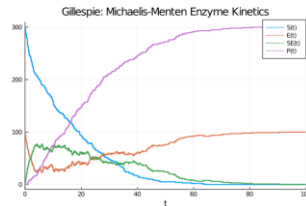


# PACKAGES

- ModelingToolkit.jl
  - Symbolic math ~1000 times faster than SymPy



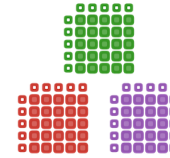
- DifferentialEquations.jl
  - Solve ODEs, PDEs, SDEs, DAEs, more
  - Fastest implementation available



- JuMP.jl
  - Symbolic interface for optimization

```
6 @constraint(model, 4f + 2s <= 4800)
7 @constraint(model, f + s <= 1750)
8 @constraint(model, 0 <= f <= 1000)
9 @constraint(model, 0 <= s <= 1500)
```

- DataFrames.jl
  - Efficient datasets (like Pandas)



- Flux.jl
  - Efficient/interpretable machine learning



- DistributedArrays.jl (GPU parallelization)
  - Easy parallelization for CPU cores/threads and GPUs



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# WHY JUMP.JL?



- High-level Syntax

- Novice friendly

```
6 @constraint(model, 4f + 2s <= 4800)
7 @constraint(model, f + s <= 1750)
8 @constraint(model, 0 <= f <= 1000)
9 @constraint(model, 0 <= s <= 1500)
```

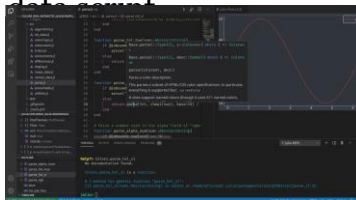
- Solver Library

- Rapidly try different solvers



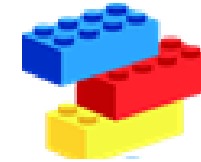
- Programmable

- Easy embed in a data context



- Extensible

- Can be extended for advanced techniques



- Open Source

- It's free!



- Performance

- Highly performant (it's Julia)



# JUMP.JL 1.0

- After nearly 10 years, JuMP.jl is now at version 1.0
- “By releasing JuMP 1.0.0, the core contributors are ensuring that all code you write using a 1.x.y release of JuMP will continue to work” until 2.0.0

[Home](#) [Documentation](#) [Community](#) ✓

## JuMP 1.0.0 is released

[releases](#) · 24 Mar 2022

*Authors: Miles Lubin, Oscar Dowson, Iain Dunning, Joaquim Dias Garcia, Joey Huchette, and Benoît Legat.*

We are happy to announce the release of JuMP 1.0!

Nearly 10 years in the making, the release of JuMP 1.0 represents a major milestone in the history of JuMP. It also represents a stable platform from which we can continue to build new and exciting features for a long time into the future.

In this post we explain what “going 1.0” means for us now and what it will mean for the future.

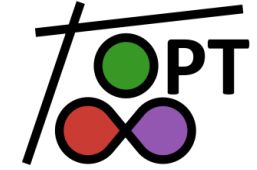
If it’s been a while since you tried JuMP, read our [release notes](#) to review some recently added features. We’ve also put a lot of effort into revising our [documentation](#), and there are a range of [getting started tutorials](#) for new users.



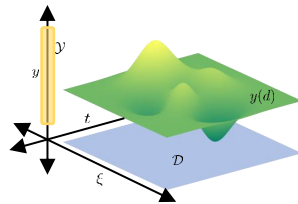
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# WHY INFINITEOPT.JL?



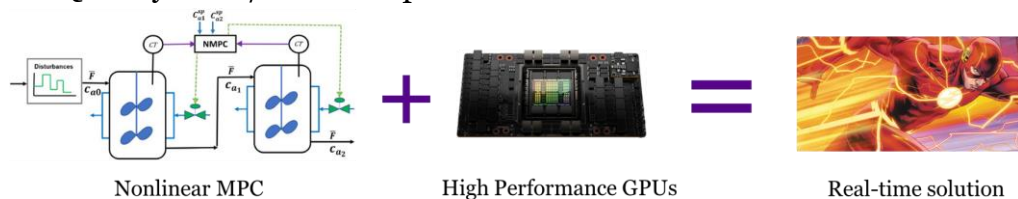
- Unifying Abstraction
  - Captures a wide breadth of problem classes



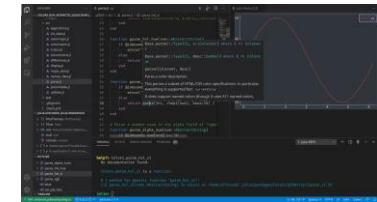
- High-level Syntax
  - Compactly express problems

```
@constraint(m, d(yb, t) == 2yb^2 + ya - z[1])
@constraint(m, yb ≤ yc * U)
@constraint(m, E(yc, ξ) ≥ α)
@constraint(m, ya(0) + z[2] == β)
```

- Performance
  - Quickly build/solve complex models at scale



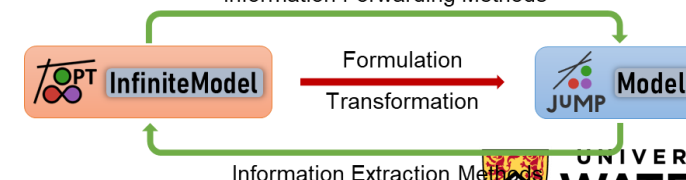
- Built on JuMP.jl
  - Draw from an extensive suite of solvers



- Open Source
  - It's free!



- Enables Accessible Research
  - Implement advanced methods for general users

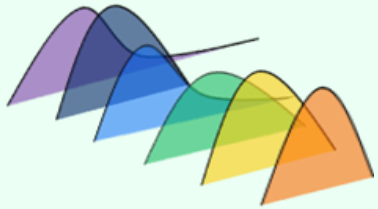


UNIVERSITY OF  
**WATERLOO**

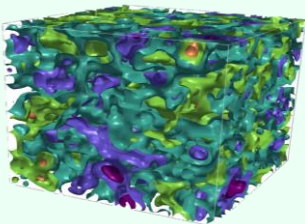
FACULTY OF  
ENGINEERING

# UNIFYING ABSTRACTION

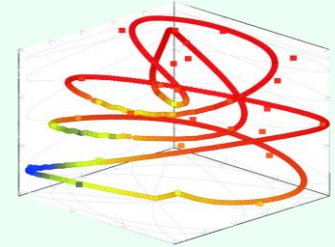
## Stochastic Optimization



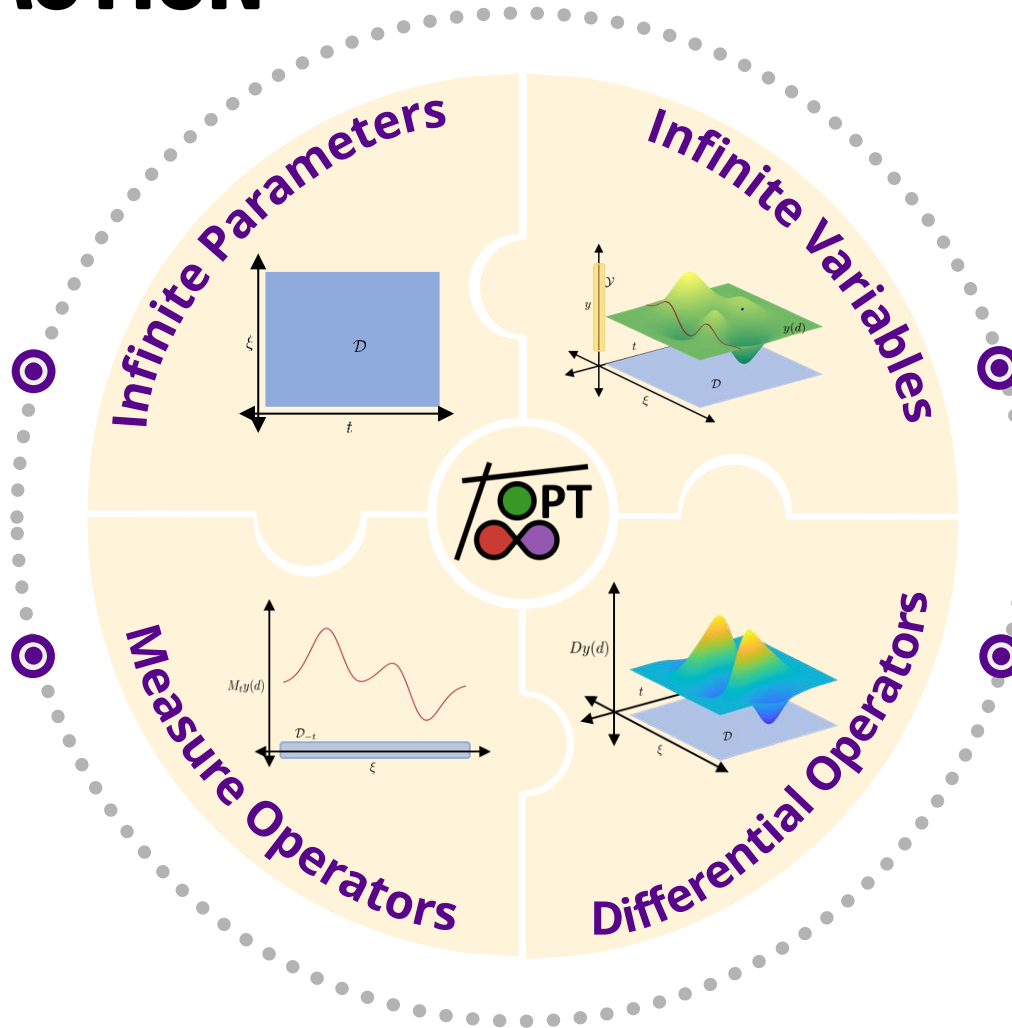
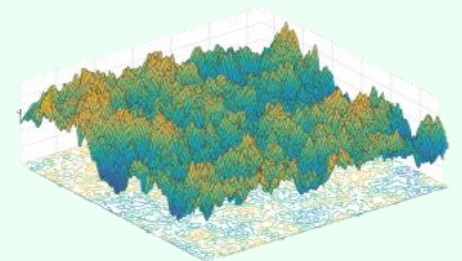
## Space-Time Optimization



## Dynamic Optimization



## Combinations





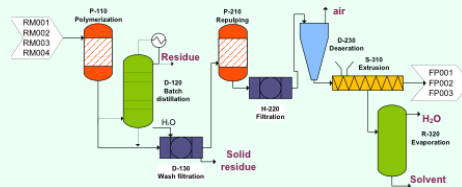
# APPLICATIONS

## Unifying Abstraction

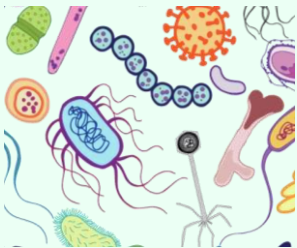
### Dynamic Optimization (Time)



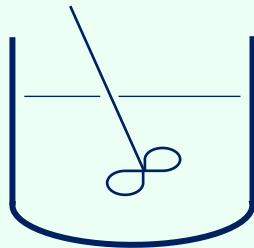
Autonomous  
Vehicles



Process Control

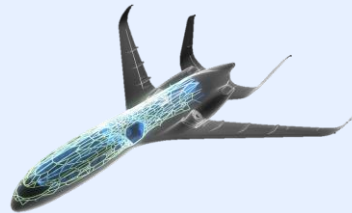


Microbial  
Communities

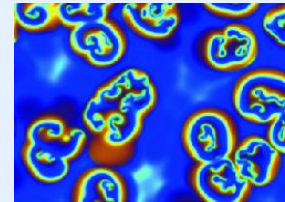


Reaction  
Systems

### PDE-Constrained Optimization (Space-Time)



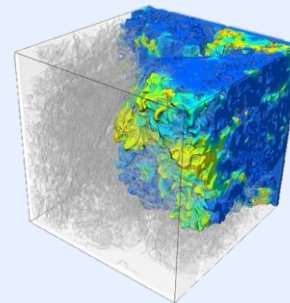
Structural  
Design



Diffusive  
Systems



Wildfire  
Simulation

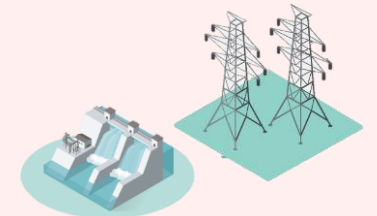


Complex Fluids

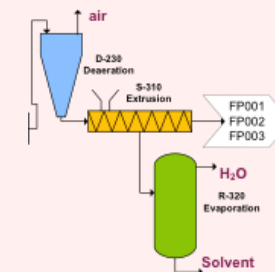
### Stochastic Optimization (Uncertainty)



Pharmaceutical  
Production



Optimal  
Power Flow



Process Design



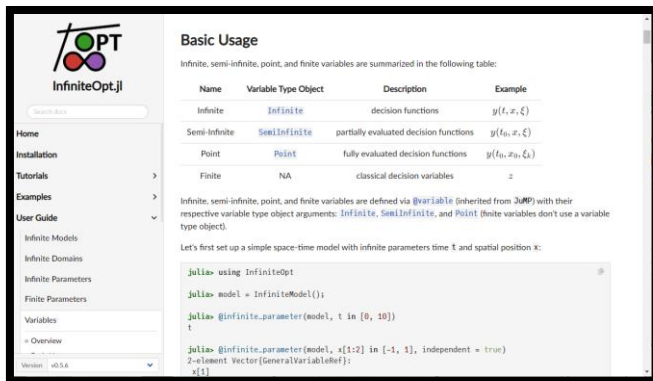
Investment  
Planning



# InfiniteOpt

## Why is it Different?

- Implements **unifying abstraction**
  - Models a wide range of problems
  - Leverages structure to **accelerate solutions**
- Implemented in **julia**
  - Enables **intuitive** symbolic expressions
  - Highly **performant**
- Extensive resources**
  - Documentation, tutorials, examples, forum, short courses, videos



Try it @ <https://github.com/infiniteopt/InfiniteOpt.jl>



## Intuitive Modeling API

$$\frac{\partial y_b(t, \xi)}{\partial t} = 2y_b(t, \xi)^2 + y_a(t) - z_1$$

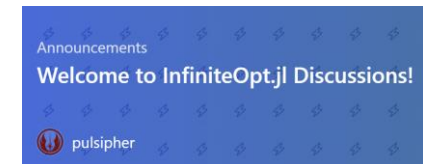
$$\mathbb{E}_\xi [y_c(t, \xi)] \geq \alpha$$

$$y_a(0) + z_2 = \beta$$

```
@constraint(m, ∂(yb, t) == 2yb^2 + ya - z[1])
@constraint(m, E(yc, ξ) ≥ α)
@constraint(m, ya(0) + z[2] == β)
```

## Impact

- 1000s of downloads
- Use cases in **diverse disciplines**
  - e.g., evolutionary biology, rocketry, economics, autonomous vehicles



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# MODELING SYNTAX

- Initialize the **model**

```
using InfiniteOpt, Distributions, Ipopt
m = InfiniteModel(Ipopt.Optimizer)
```

- Add the **infinite parameters**

$$t \in [t_0, t_f] \quad \xi \sim \mathcal{N}(\mu, \Sigma)$$

```
@infinite_parameter(m, t ∈ [t0, tf], num_supports = 10)
@infinite_parameter(m, ξ[1:10] ~ MvNormal(μ, Σ))
```

- Add **variables** and their domain constraints

$$y_a(t) \in \mathbb{R}_+ \quad y_b(t, \xi) \in \mathbb{R}_+ \quad y_c(\xi) \in \{0, 1\} \quad z \in \mathbb{Z}^2$$

```
@variable(m, ya ≥ 0, Infinite(t))
@variable(m, yb ≥ 0, Infinite(t, ξ))
@variable(m, yc, Infinite(ξ), Bin)
@variable(m, z[1:2], Int)
```

- Define the **objective**

$$\min_{y_a(t), y_b(t, \xi), y_c(\xi), z} \int_{t \in \mathcal{D}_t} y_a(t)^2 + 2\mathbb{E}_\xi[y_b(t, \xi)] dt$$

```
@objective(m, Min, ∫(ya^2 + 2 * E(yb, ξ), t))
```

- Add the **constraints**

$$\frac{\partial y_b(t, \xi)}{\partial t} = 2y_b(t, \xi)^2 + y_a(t) - z_1, \quad \forall t \in \mathcal{D}_t, \xi \in \mathcal{D}_\xi$$
$$\mathbb{P}(y_b(t, \xi) \leq 0) \geq \alpha, \quad \forall t \in \mathcal{D}_t$$
$$y_a(0) + z_2 = \beta$$

```
@constraint(m, ∂(yb, t) == 2yb^2 + ya - z[1])
@constraint(m, yb ≤ yc * U)
@constraint(m, E(yc, ξ) ≥ α)
@constraint(m, ya(0) + z[2] == β)
```

- Transform and Solve**

```
optimize!(m)
opt_objective = objective_value(m)
```



# WHAT MODELING OBJECTS DOES IT SUPPORT?

## InfiniteOpt Specific

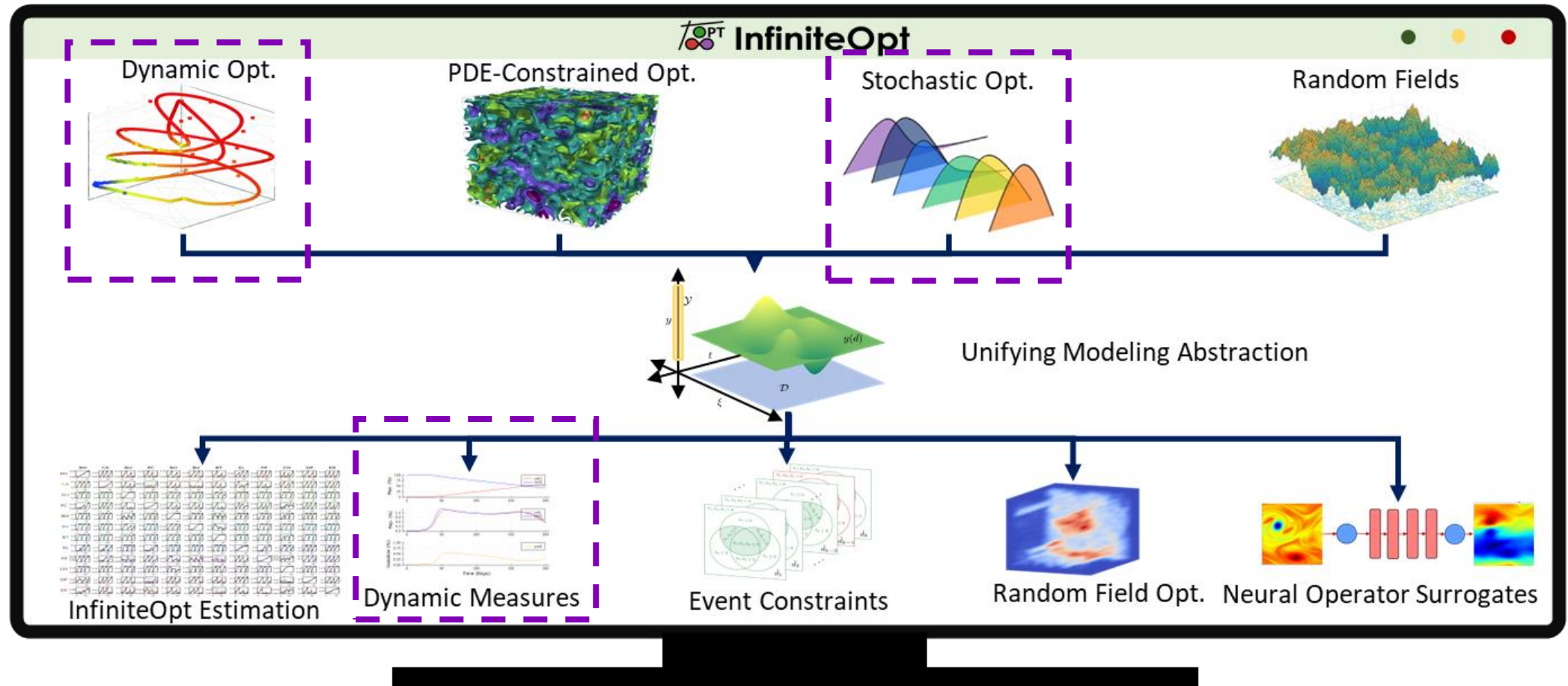
- Infinite parameters
  - Any distribution from Distributions.jl
  - Cartesian domains (e.g., time/space)
- Integrals
- Expectations
- Ordinary differential equations
- Partial differential equations
- Chance constraints
- Infinite disjunctions and logic constraints

## Inherited from JuMP

- Affine/quadratic/nonlinear/mixed-integer models
- Constraint programming
- Vector-valued constraints
- Conic constraints
- Complementarity constraints
- Multi-objectives
- Indicator constraints
- Many many more...



# INNOVATION VIA CROSS POLLINATION



J. L. Pulsipher, W. Zhang, T. J. Hongisto, and V. M. Zavala. "A unifying modeling abstraction for infinite-dimensional optimization." 2022

J. L. Pulsipher, B. Davidson, and V. M. Zavala. "New Measures for Shaping Trajectories in Dynamic Optimization." 2022

J. L. Pulsipher, Hongisto, and V. M. Zavala. "Random Field Optimization." 2023

D. Ovalle, H. Perez, I. E. Grossmann, C. D. Laird, J. L. Pulsipher. "Event Constrained Programming". In progress



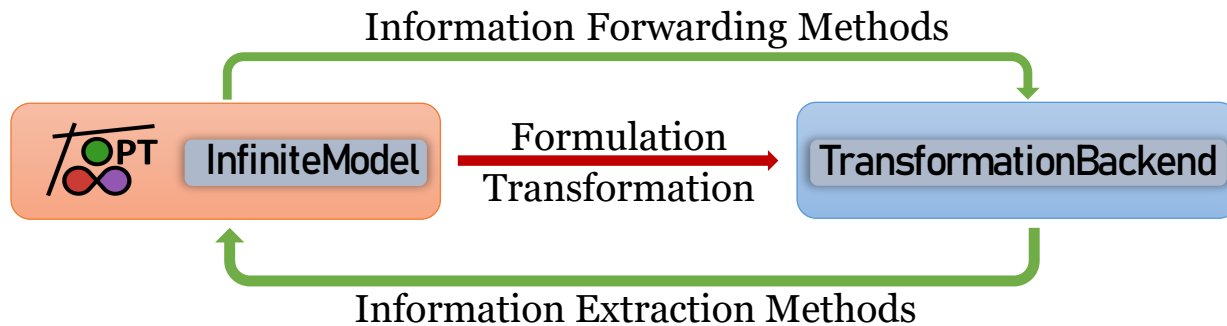
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# TRANSFORMING INFINITEOPT MODELS



## Transformation Paradigm

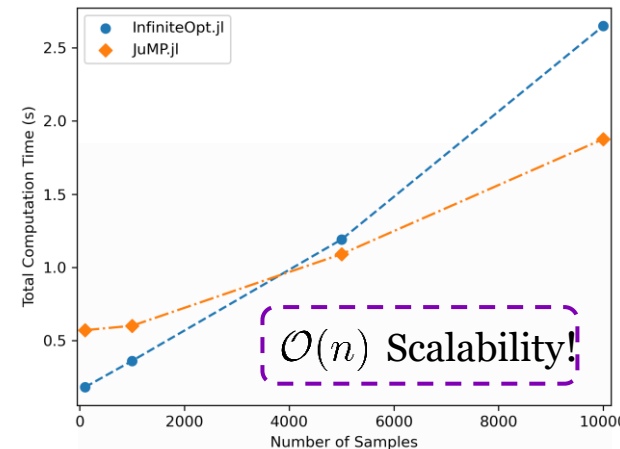


## Transformation API

- Highly extensible to **make advanced solution techniques accessible/automated**
- Detailed templates, tutorials, and docs

## Automated Transcription (via TranscriptionOpt)

- Many **derivative/measure approximations**
  - Orthogonal collocation, Gauss quadrature, etc.
- **Performant**



InfiniteOpt.jl



**Automatic Transcription**

JuMP.jl



**Manual Transcription**



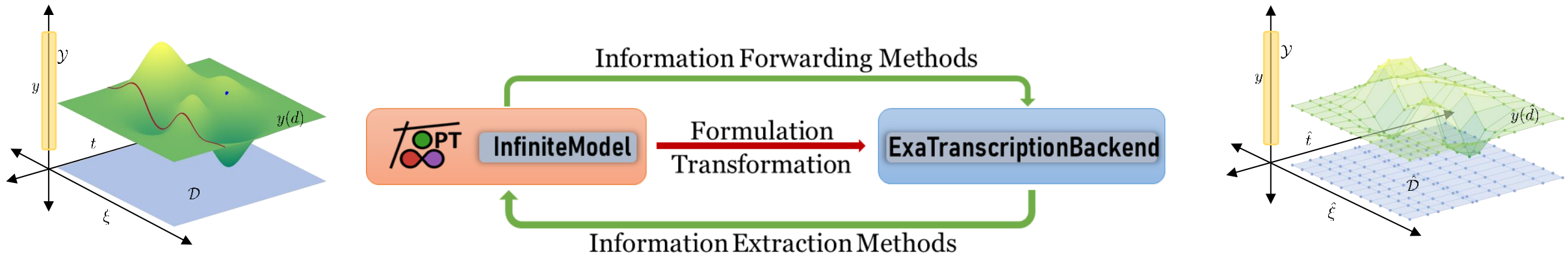
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING



# ACCELERATING SOLUTION

- Leverage repeated structure to **drastically reduce model creation time**
- Enables the **fastest general NLP-InfiniteOpt** solver available



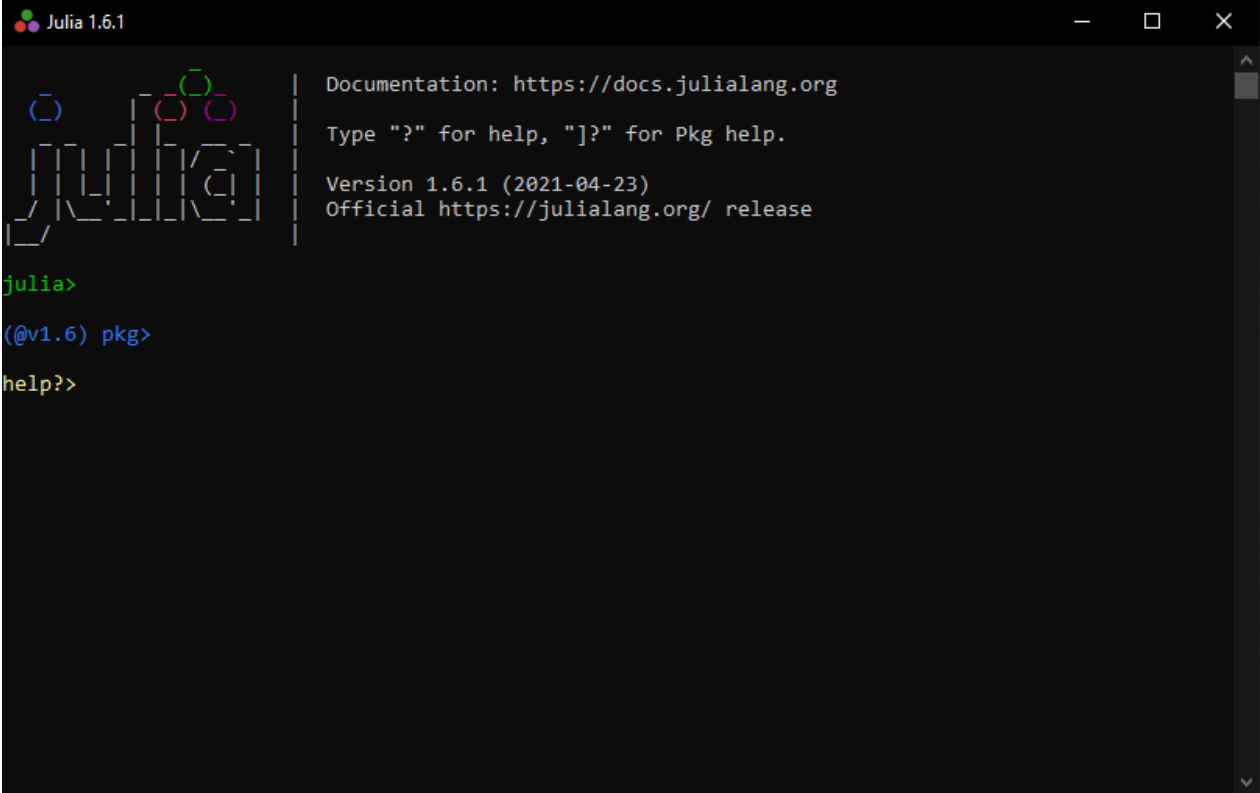




## A Practical Introduction

# THE REPL

- What is it?
  - Read-Evaluate-Print-Loop
  - Executes commands, code, scripts
- What can I do with
  - Run quick commands
  - Run entire scripts
  - Play around
  - Get help
  - This is analogous to the Python kernel
- Package management
  - Use the package mode
  - More on this later



```
Julia 1.6.1

Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.

Version 1.6.1 (2021-04-23)
Official https://julialang.org/ release

julia>
(@v1.6) pkg>
help?>
```



# TO THE TUTORIAL!

- Open the Part 1 Jupyter notebook in Colab

colab



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# PACKAGES

- Libraries of abilities
  - Nearly 7,000 packages are available
  - Access a wealth of functions and capabilities
- Package manager
  - Use the Pkg.jl interface in the REPL
  - Manages dependencies
- Environments
  - An independent collection of packages
  - Stops packages from stepping on each others toes
  - See documentation for details
- Importing
  - Use the `using` or `import` keywords

```
Julia 1.6.1

(@v1.6) pkg> add Plots
Updating registry at `C:\Users\pulsipher\.julia\registries\General`
Updating git-repo `https://github.com/JuliaRegistries/General.git`
Resolving package versions...
Installing ImageMagick_jll - v6.9.12+1
Installing Ogg_jll - v1.3.5+1
Downloaded artifact: Ogg
Downloaded artifact: ImageMagick
No Changes to `C:\Users\pulsipher\.julia\environments\v1.6\Project.toml`
Updating `C:\Users\pulsipher\.julia\environments\v1.6\Manifest.toml`
[c73af94c] ↑ ImageMagick_jll v6.9.12+0 → v6.9.12+1
[e7412a2a] ↑ Ogg_jll v1.3.5+0 → v1.3.5+1
Precompiling project...
106 dependencies successfully precompiled in 183 seconds (257 already precompiled)

(@v1.6) pkg> status
Status `C:\Users\pulsipher\.julia\environments\v1.6\Project.toml`
[1520ce14] AbstractTrees v0.3.4
[9961bab8] Cbc v0.9.0
[e2554f3b] Clp v0.9.0
[864edb3b] DataStructures v0.18.10
[0c46a032] DifferentialEquations v6.19.0
[31c24e10] Distributions v0.25.14
[e30172f5] Documenter v0.27.10
[442a2c76] FastGaussQuadrature v0.4.7
[5789e2e9] FileIO v1.11.2
[e4b2fa32] GaussianRandomFields v2.1.4
[2e9cd046] Gurobi v0.10.0
[f67ccb44] HDF5 v0.15.7
[7073ff75] IJulia v1.23.2
[02fcd773] ImageTransformations v0.8.13
[916415d5] Images v0.24.1
[20393b10] InfiniteOpt v0.5.1 `C:\Users\pulsipher\.julia\dev\InfiniteOpt`
[a98d9a8b] Interpolations v0.13.4
[b6b21f68] Ipopt v0.8.0
[682c06a0] JSON v0.21.2
[4076af6c] JuMP v0.22.0
[b964fa9f] LaTeXStrings v1.3.0
[1d6d02ad] LeftChildRightSiblingTrees v0.1.2
[98b081ad] Literate v2.9.4
[b8f27783] MathOptInterface v0.10.5
[d8a4904e] MutableArithmetics v0.3.1
[14b8a8f1] PkgTemplates v0.7.25
[91a5bcdd] Plots v1.23.6
[92933f4c] ProgressMeter v1.7.1
[438e738f] PyCall v1.92.5
[d330b81b] PyPlot v2.10.0
[189a3867] Reexport v1.2.2
[295af30f] Revise v3.1.20
[276daf66] SpecialFunctions v1.8.1
[f094767] Suppressor v0.2.0
[0c5d862f] Symbolics v4.2.1

(@v1.6) pkg>
```



# PLOTTING

- Plots.jl
  - Efficient go to
  - The documentation is terrible ☐
- PyPlot.jl
  - Simply maps to matplotlib.pyplot
  - Imports all the all the pyplot functions
- Makie.jl
  - Native to Julia
  - Good documentation
  - Can be slow sometimes
- Others
  - GadFly.jl (pretty good 2D library)
  - PGFPlots.jl (plots for LaTeX)
  - UnicodePlots.jl (plot directly in the REPL)

```
Julia 1.6.1

julia> using Plots

julia> y(t) = sin(t) * cos(t)
y (generic function with 1 method)

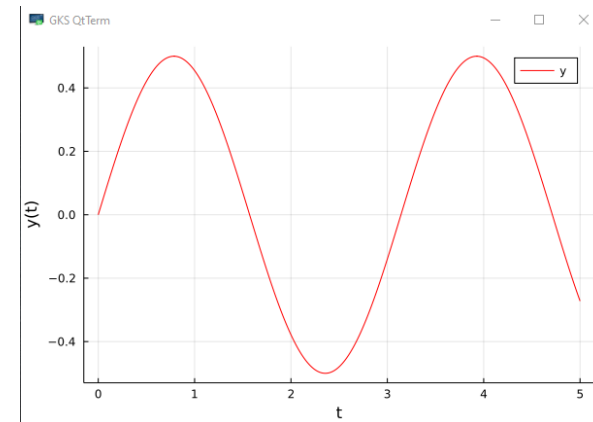
julia> ts = 0:0.01:5
0.0:0.01:5.0

julia> plot(ts, y.(ts), label = "y", color = "red")

julia> xlabel!("t")

julia> ylabel!("y(t)")

julia>
```



# BASIC PARALLEL COMPUTING

- Distributed.jl
  - Provides the basic CPU parallel computing methods
  - Add parallel resources via `addprocs`
  - Parallel for loops using `@distributed`
  - Other packages like MPI.jl take this further
  - To use arrays look at DistributedArrays.jl
  - Good for “larger jobs” (increased memory)
- Multi-threading via Threads
  - Operate on for loops with `Threads.@threads`
  - Must start Julia with the # of threads `$ Julia -threads 4`
  - Use cautiously (shared memory)
- GPU computing
  - Run Julia code on GPUs
  - See [juliagpu.org](http://juliagpu.org) for info

```
Julia 1.6.1

julia> using Distributed

julia> nprocs()
1

julia> addprocs(3)
3-element Vector{Int64}:
 2
 3
 4

julia> nprocs()
4

julia> function parallel_sum_even(x)
           result = @distributed (+) for i in 1:x
                               iseven(i)
                           end
           return result
       end
parallel_sum_even (generic function with 1 method)

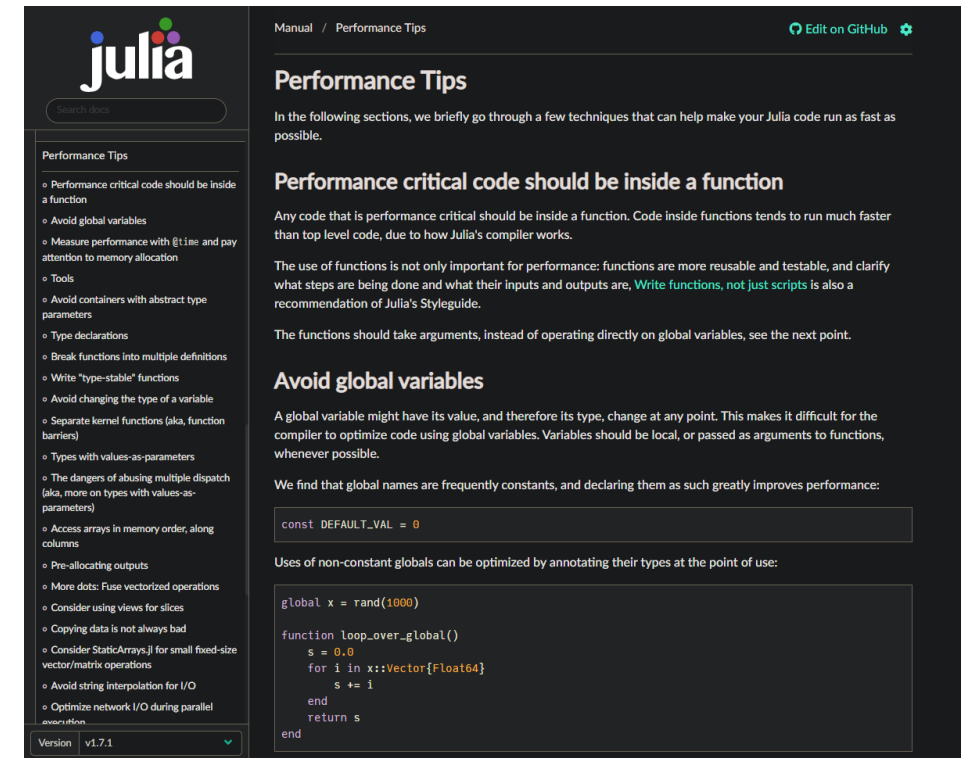
julia> parallel_sum_even(1000)
500

julia>
```



# PERFORMANCE

- The performance tips page
  - <https://docs.julialang.org/en/v1/manual/performance-tips/>
- Julia code *can be* C-fast
- Can be slow if we have poorly structured code
- Common practices
  - Preallocate instead of appending to arrays
  - Access arrays in memory order
  - Avoid ambiguous types
  - Use function overloading



The screenshot shows the Julia Performance Tips page. On the left is a sidebar with the Julia logo and a search bar. Below it is a list of performance tips topics. The main content area is titled 'Performance Tips' and includes an 'Edit on GitHub' link. It contains three sections: 'Performance critical code should be inside a function', 'Avoid global variables', and a code example for a loop over a global variable.

Manual / Performance Tips [Edit on GitHub](#)

## Performance Tips

In the following sections, we briefly go through a few techniques that can help make your Julia code run as fast as possible.

### Performance critical code should be inside a function

Any code that is performance critical should be inside a function. Code inside functions tends to run much faster than top level code, due to how Julia's compiler works.

The use of functions is not only important for performance: functions are more reusable and testable, and clarify what steps are being done and what their inputs and outputs are. [Write functions, not just scripts](#) is also a recommendation of Julia's Styleguide.

The functions should take arguments, instead of operating directly on global variables, see the next point.

### Avoid global variables

A global variable might have its value, and therefore its type, change at any point. This makes it difficult for the compiler to optimize code using global variables. Variables should be local, or passed as arguments to functions, whenever possible.

We find that global names are frequently constants, and declaring them as such greatly improves performance:

```
const DEFAULT_VAL = 0
```

Uses of non-constant globals can be optimized by annotating their types at the point of use:

```
global x = rand{1000}

function loop_over_global()
    s = 0.0
    for i in x::Vector{Float64}
        s += i
    end
    return s
end
```

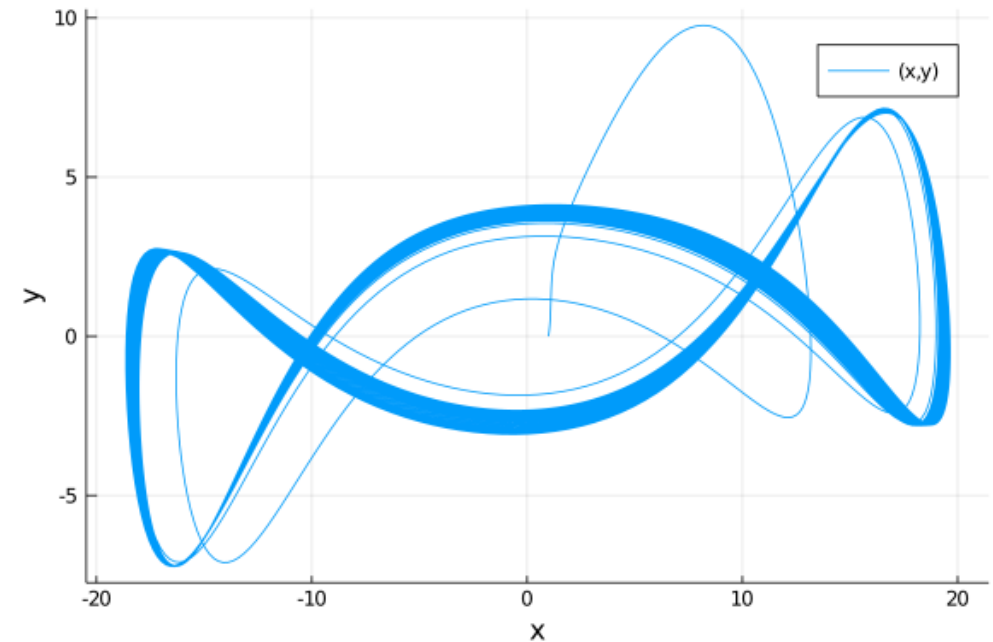
Version v1.7.1





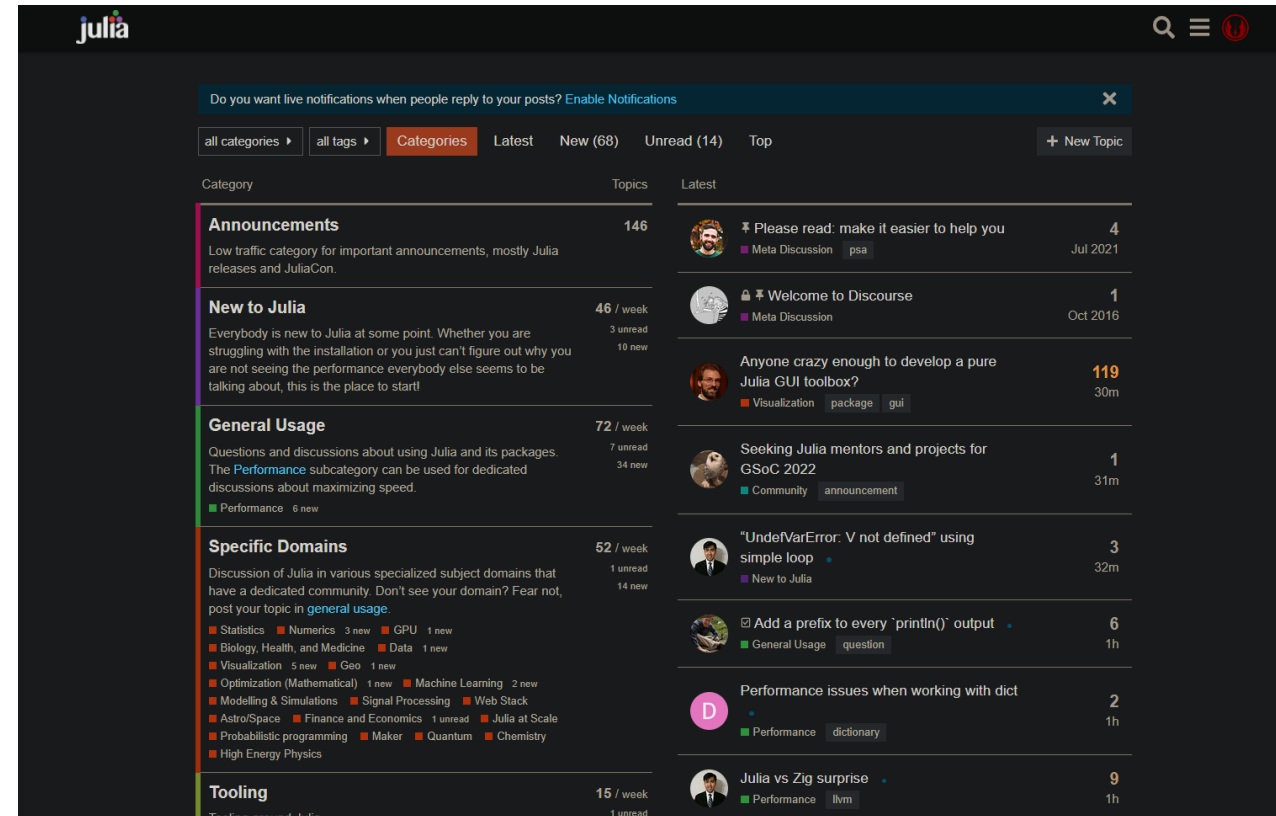
# OTHER USEFUL THINGS

- File reading and writing
  - Delimited files via DelimitedFiles (built-in)
    - Text files and basic CSVs
  - CSVs via CSV.jl
  - Store/read dictionaries via JSON.jl
  - Tables via DataFrames.jl
- Math stuff
  - Symbolic math with Symbolics.jl and ModelingToolkit
  - Differential equations with DifferentialEquations.jl
  - Statistics with Statistics.jl and Distributions.jl
- Machine learning
  - Flux.jl
  - Tensorflow.jl and ScikitLearn.jl
  - Knet.jl



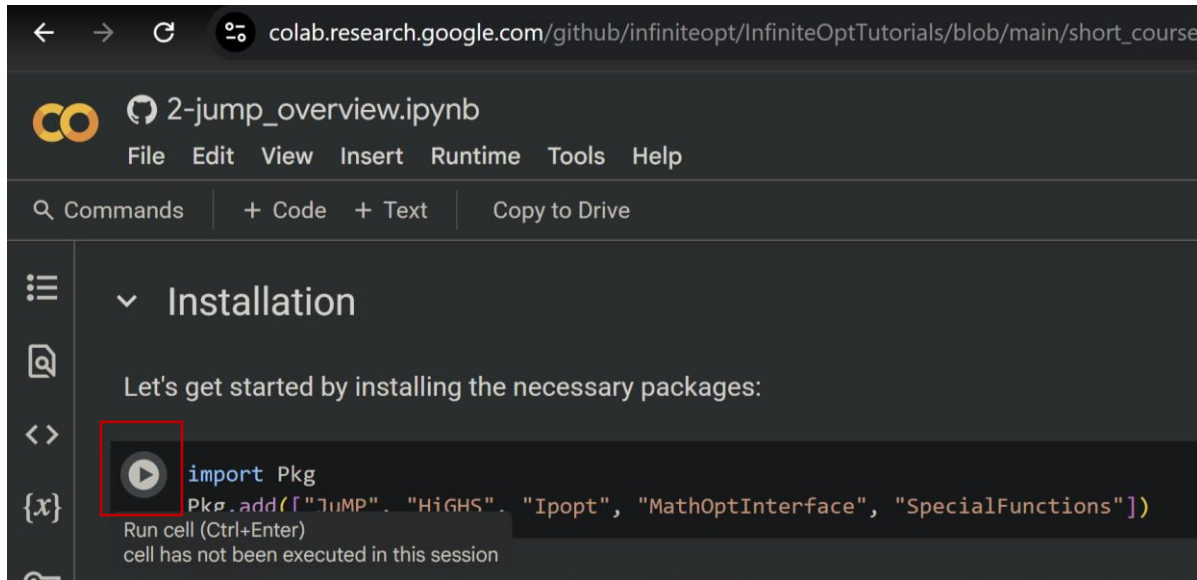
# RESOURCES

- Learn Julia in Y Minutes
  - <https://learnxinyminutes.com/docs/julia/>
- Julia's documentation
  - <https://docs.julialang.org/en/v1/>
- Julia's Discourse forum
  - <https://discourse.julialang.org/>
- Julia's YouTube channel
  - <https://www.youtube.com/user/julialanguage>
  - Check out the tutorials and JuliaCon talks
- Google it!



# PART 2 SET UP

- Before the break, please open Part 2 and run the first cell to install packages



```
import Pkg
Pkg.add(["JUMP", "HiGHS", "Ipopt", "MathOptInterface", "SpecialFunctions"])
```



# 15 MINUTE BREAK TIME



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING



An Overview

# MODELING IN JUMP.JL

$$\begin{array}{ll}\max_{f,s} & 12f + 9s \\ \text{s.t.} & 4f + 2s \leq 4800 \\ & f + s \leq 1750 \\ & 0 \leq f \leq 1000 \\ & 0 \leq s \leq 1500\end{array}$$

- Initialize **model**

```
1 using JuMP, Gurobi
2 model = Model{Gurobi.Optimizer}
```

- Define **variables**

```
3 @variable(model, f)
4 @variable(model, s)
```

- Define **objective**

```
5 @objective(model, Max, 12f + 9s)
```

- Define **constraints**

```
6 @constraint(model, 4f + 2s <= 4800)
7 @constraint(model, f + s <= 1750)
8 @constraint(model, 0 <= f <= 1000)
9 @constraint(model, 0 <= s <= 1500)
```

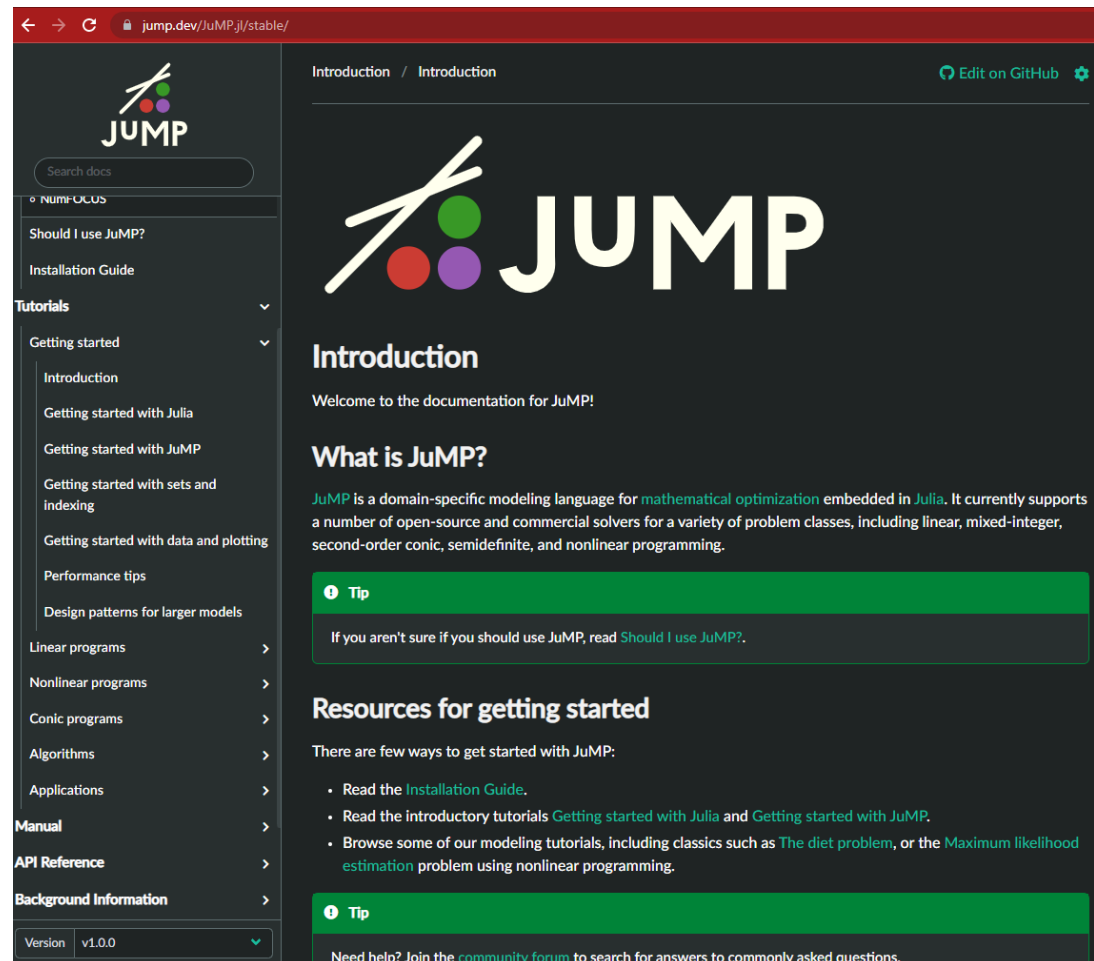
- **Solve**

```
10 optimize!(model)
11 profit = objective_value(model)
12 f_best = value(f)
13 s_best = value(s)
```



# LEARNING JUMP.JL

- The documentation is full of very useful tutorials



The screenshot shows the JuMP.jl documentation website. The browser address bar displays `jump.dev/JuMP.jl/stable/`. The left sidebar contains a navigation menu with the following items: **JUMP** (with a search bar), **NUMFLOUS**, **Should I use JuMP?**, **Installation Guide**, **Tutorials** (expanded), **Getting started** (expanded), **Introduction**, **Getting started with Julia**, **Getting started with JuMP**, **Getting started with sets and indexing**, **Getting started with data and plotting**, **Performance tips**, **Design patterns for larger models**, **Linear programs**, **Nonlinear programs**, **Conic programs**, **Algorithms**, **Applications**, **Manual**, **API Reference**, and **Background Information**. The main content area is titled **Introduction** and features the JuMP logo. It includes a welcome message, a section titled **What is JuMP?** explaining it as a domain-specific modeling language for mathematical optimization embedded in Julia, and a **Resources for getting started** section with links to the Installation Guide, introductory tutorials, and modeling tutorials. Two green tip boxes are present: one suggesting to read 'Should I use JuMP?' and another encouraging users to join the community forum for help.

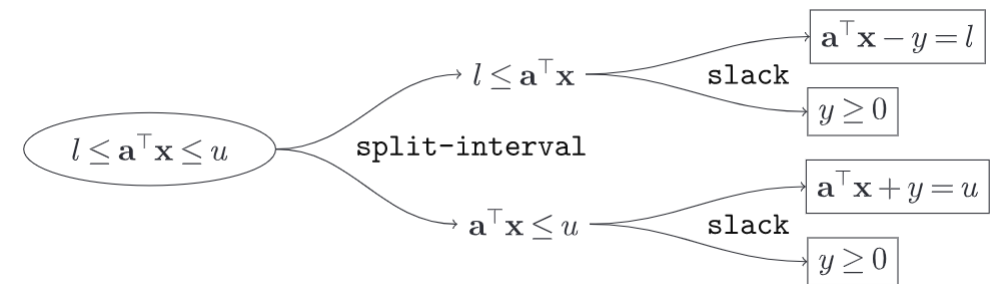
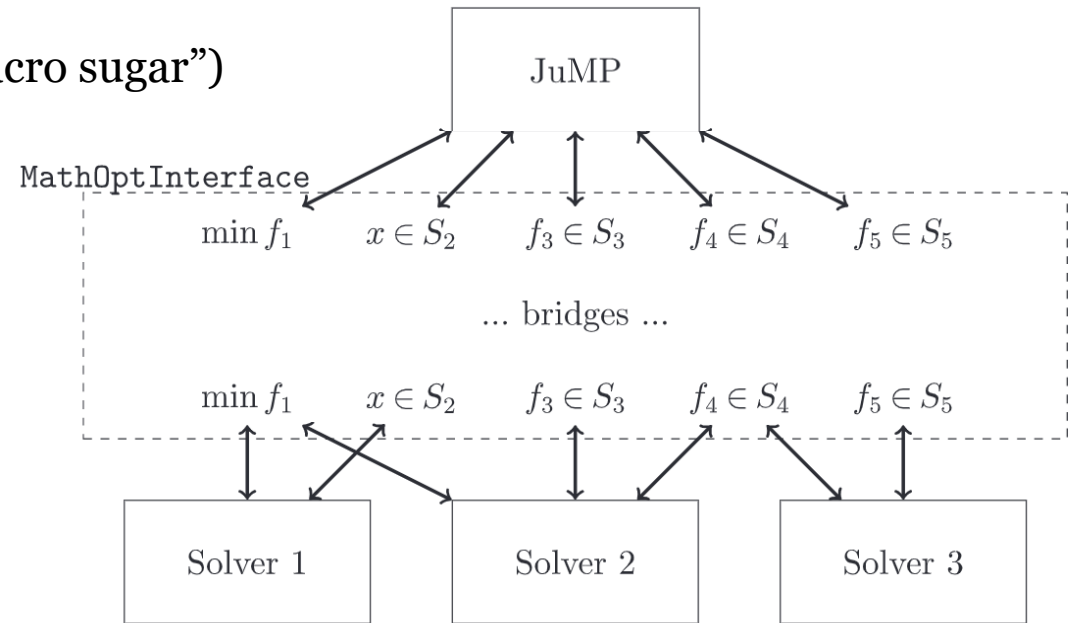


UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# JUMP.JL ARCHITECTURE

- JuMP.jl provides the high-level interface (the “macro sugar”)
- Model is stored in MathOptInterface.jl model
  - Constraints are of the form *function* in *set*
- Bridges convert constraints into form that solvers support
  - Solvers don't explicitly support every *function-set* combination
- MathOptInterface.jl preserves the constraint mappings
- Solvers interface with MathOptInterface.jl





# TO THE TUTORIAL!

- Open the Part 2 Jupyter notebook in Colab

colab

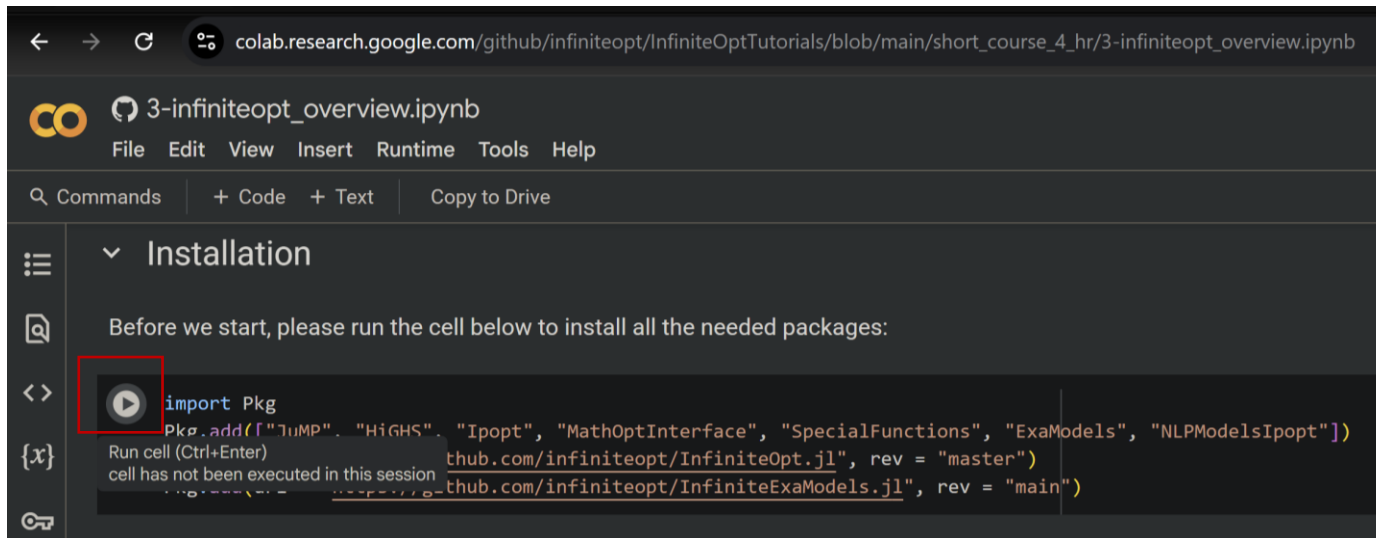


UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# PART 3 SET UP

- Before the break, please open Part 3 and run the first cell to install packages



colab.research.google.com/github/infiniteopt/InfiniteOptTutorials/blob/main/short\_course\_4\_hr/3-infiniteopt\_overview.ipynb

3-infiniteopt\_overview.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Copy to Drive

Installation

Before we start, please run the cell below to install all the needed packages:

```
import Pkg
Pkg.add(["JUMP", "HiGHS", "Ipopt", "MathOptInterface", "SpecialFunctions", "ExaModels", "NLModelsIpopt"])
Pkg.update(["JUMP", "HiGHS", "Ipopt", "MathOptInterface", "SpecialFunctions", "ExaModels", "NLModelsIpopt"])
Pkg.set_package_registry("https://github.com/infiniteopt/InfiniteOpt.jl", rev = "master")
Pkg.set_package_registry("https://github.com/infiniteopt/InfiniteExaModels.jl", rev = "main")
```

Run cell (Ctrl+Enter)  
cell has not been executed in this session



# 15 MINUTE BREAK TIME



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING



An Overview

# DYNAMIC OPTIMIZATION

## Dynamic Optimization

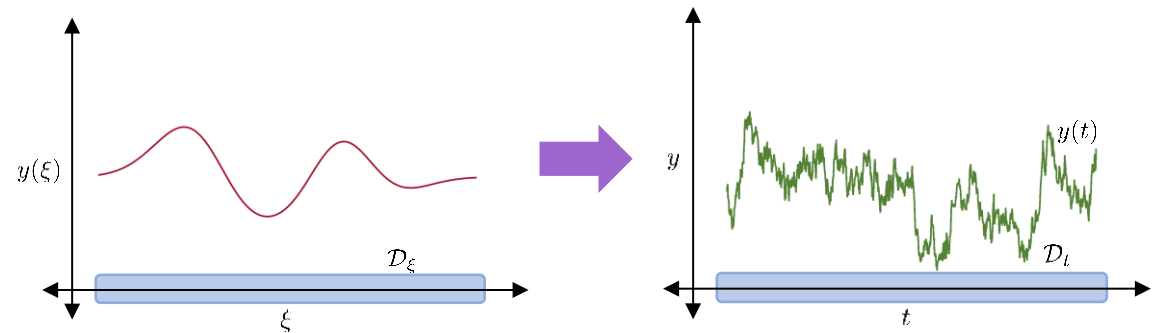
$$\begin{aligned} \min_{\mathbf{y}(t)} \quad & \int_{t \in \mathcal{D}_t} f(\mathbf{y}(t), t) dt \\ \text{s.t.} \quad & g(\mathbf{y}(t), t) \leq 0, \quad t \in \mathcal{D}_t \end{aligned}$$

## Stochastic Optimization

$$\begin{aligned} \min_{\mathbf{y}(\xi)} \quad & \int_{\xi \in \mathcal{D}_\xi} f(\mathbf{y}(\xi), \xi) p(\xi) d\xi \\ \text{s.t.} \quad & g(\mathbf{y}(\xi), \xi) \leq 0, \quad \xi \in \mathcal{D}_\xi \end{aligned}$$

## Different problem, similar features

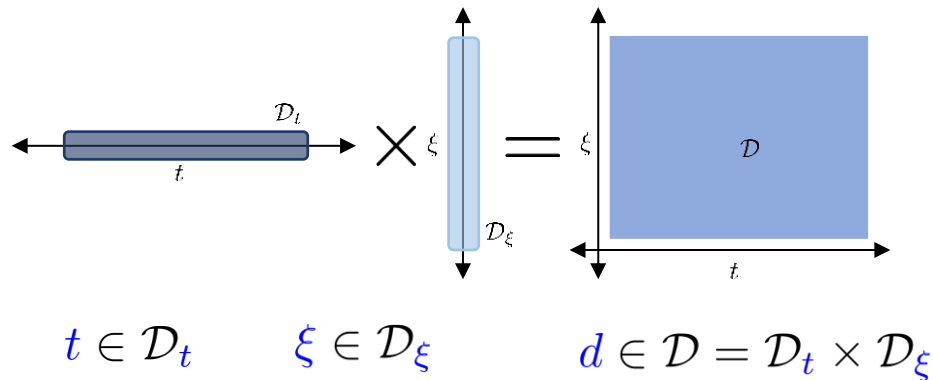
- Replace  $\xi \in \mathcal{D}_\xi$  with time  $t \in \mathcal{D}_t$
- Let  $p(\xi) = 1$
- Shape  $\mathbf{y}(t)$  instead of  $\mathbf{y}(\xi)$



# INFINITE-DIMENSIONAL OPTIMIZATION

## Infinite Parameters

- Index over **continuous domains**



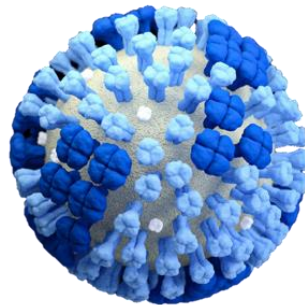
- Example:** Disease Control

- Population dynamics

$$t \in [0, t_f]$$

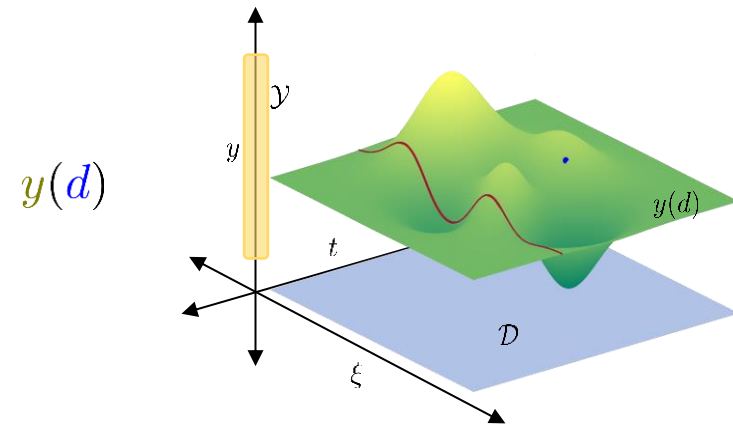
- Uncertain infection rates

$$\xi \in (-\infty, \infty) \sim \mathcal{N}(\mu, \Sigma)$$



## Infinite Variables

- Decisions** indexed by infinite parameters



- Example:** Disease Control

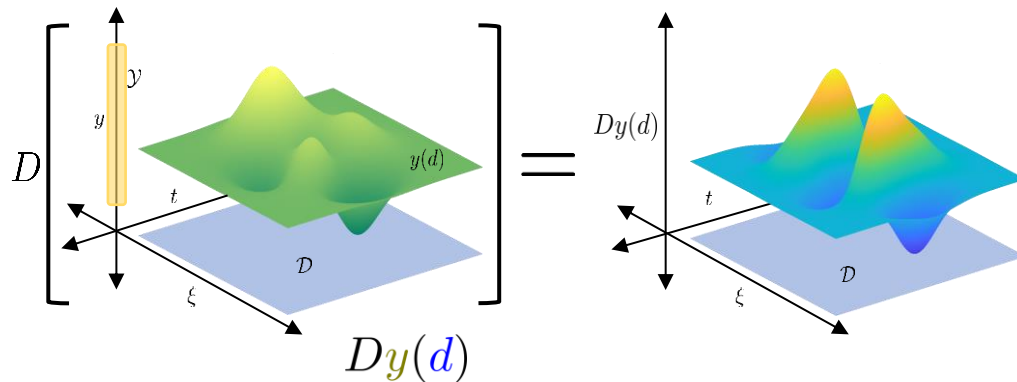
- Population of infected at a particular time and infection rate

$$y_i(t, \xi)$$

# INFINITE-DIMENSIONAL OPTIMIZATION

## Differential Operators

- Capture of **rate of change** in variables

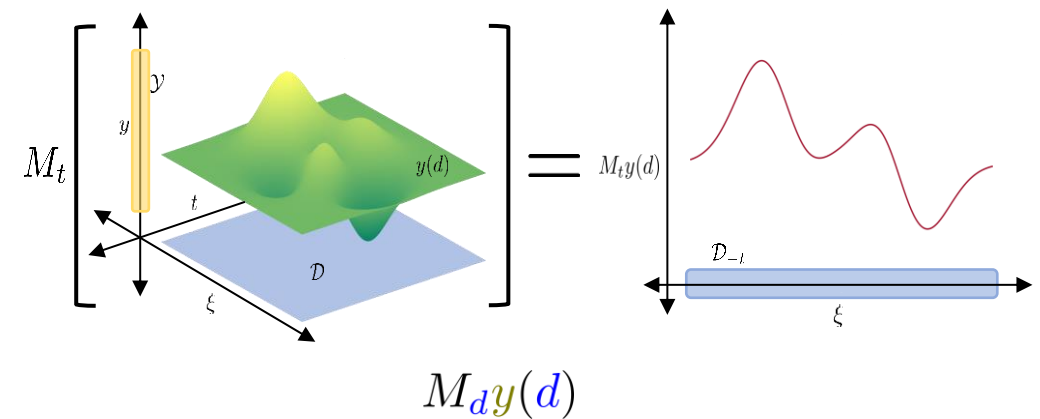


- Example:** Disease Control

- Time derivative  $\frac{\partial y_i(t, \xi)}{\partial t}$
- SEIR model  $\frac{\partial y_i(t, \xi)}{\partial t} = \xi y_e(t) - \gamma y_i(t)$

## Measure Operators

- Summarize variables** over continuous domains



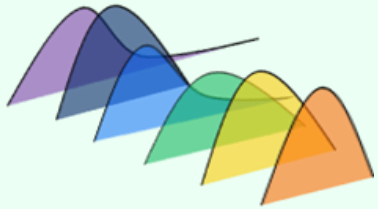
- Example:** Disease Control

- Summarize overall infections

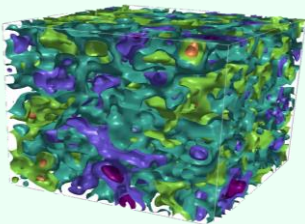
$$\int_{t \in \mathcal{D}_t} \mathbb{E}_\xi [y_i(t, \xi)] dt \quad \mathbb{E}_\xi \left[ \int_{t \in \mathcal{D}_t} y_i(t, \xi) dt \right]$$

# UNIFYING ABSTRACTION

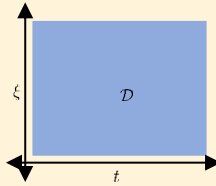
## Stochastic Optimization



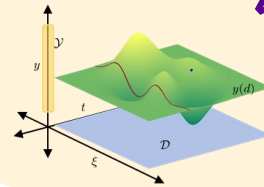
## Space-Time Optimization



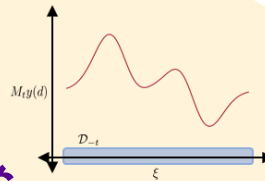
Infinite Parameters



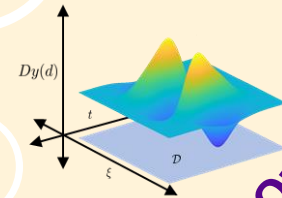
Infinite Variables



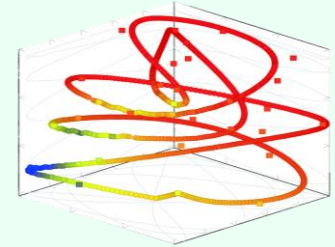
Measure Operators



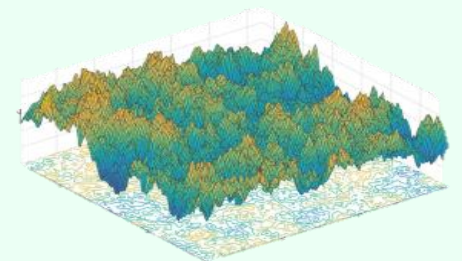
Differential Operators



## Dynamic Optimization



## Combinations



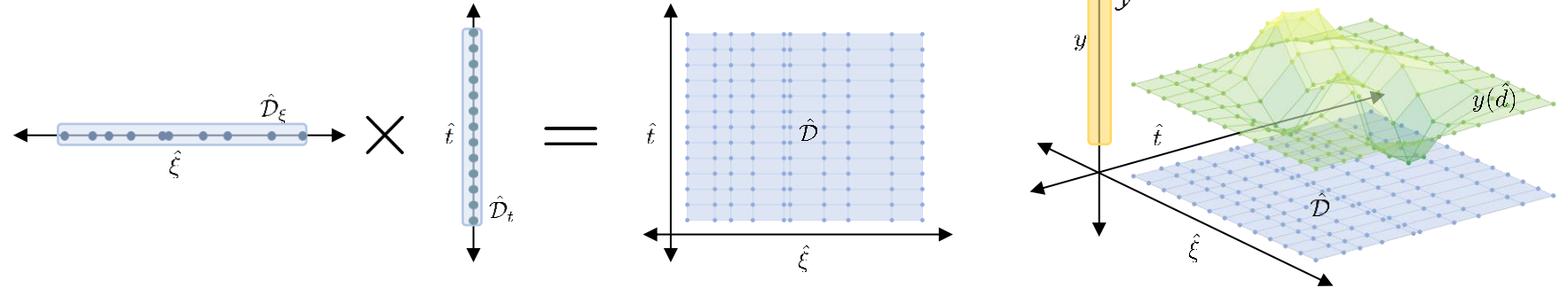


# TRANSFORMING INFINITEOPT PROBLEMS INTO FINITE ONES

## Direct Transcription

Project onto set of  
finite points  $\hat{\mathcal{D}}$

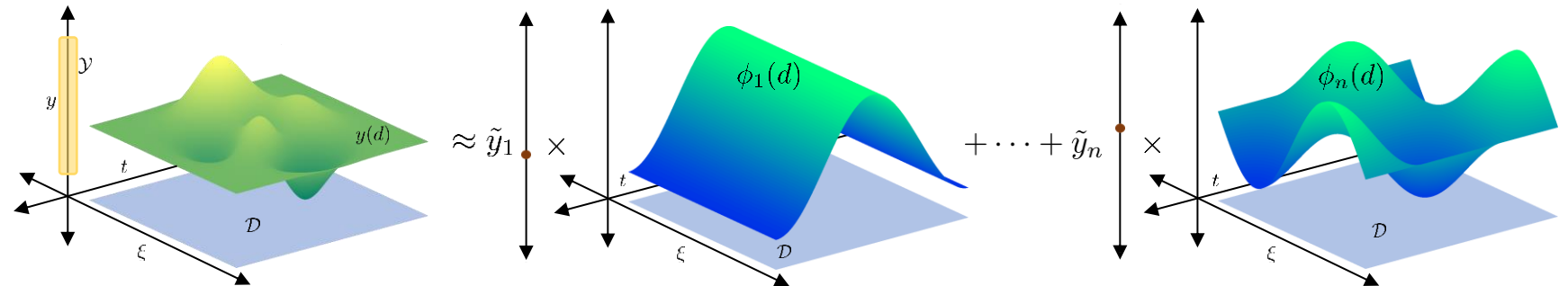
$$\hat{\mathcal{D}} := \prod_{\ell \in \mathcal{L}} \{\hat{d}_{\ell,i} : \hat{d}_{\ell,i} \in \mathcal{D}_{\ell}, i \in \mathcal{I}_{\ell}\}$$



## Method of Weighted Residuals

Project onto set of  
known **basis functions**

$$y(d) \approx \sum_{i \in \mathcal{I}} \tilde{y}_i \phi_i(d)$$

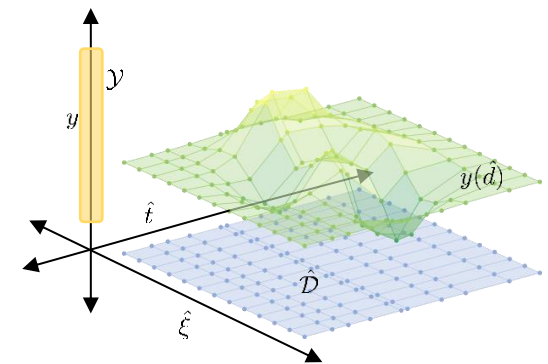
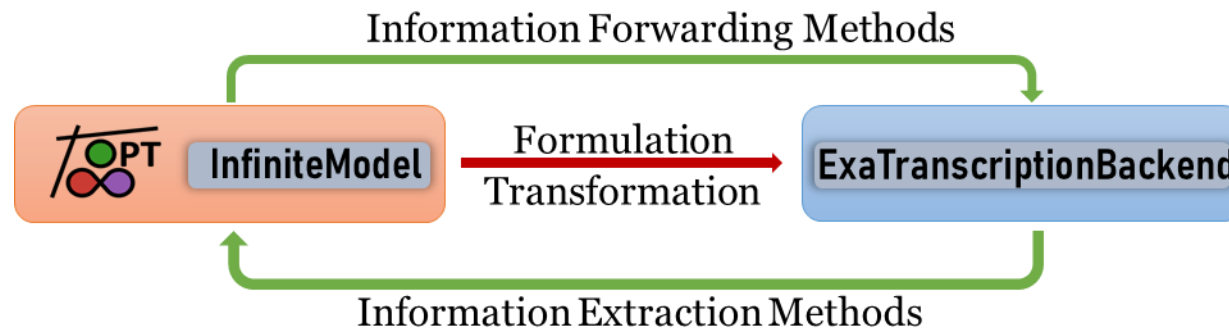
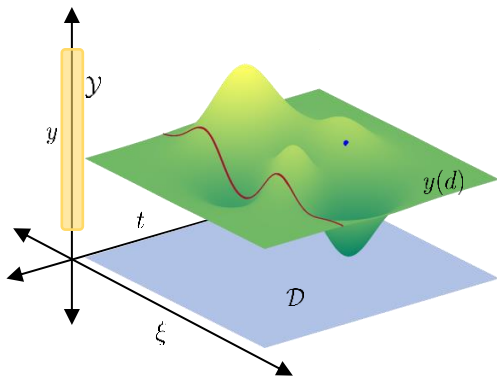


UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING

# INFINITEEXAMODELS.JL

- Bridges the gap between  **InfiniteOpt** &  **ExaModels**
- **Automates transcription** via established transformation interface
- Leverages repeated structure to **drastically reduce model creation time**
  - More efficient than manual transcription directly given to ExaModels



# IMPLEMENTATION DETAILS

- Supports the use of **JSO NLP solvers** (e.g., Ipopt, MadNLP, KNITRO)
- Defined via an `ExaTranscriptionBackend`

```
1 using InfiniteOpt, InfiniteExaModels, MadNLPGPU, CUDA
2 transform_backend = ExaTranscriptionBackend(MadNLPSolver, backend = CUDABackend())
3 model = InfiniteModel(transform_backend)
```

- Rapidly transcribes infinite model into **efficient ExaModels**
- **Model build** time is nearly **independent of the discretization size**

# BENCHMARK PROBLEMS

- Compare performance with JuMP, AMPL, MathOptSymbolicAD, ExaModels, and InfiniteExaModels
- Run on CPU with Ipopt and GPU with MadNLP

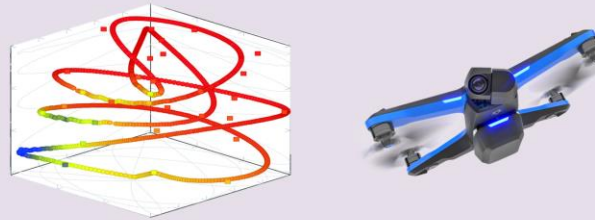
## 2-Stage Stochastic Program

- Stochastic optimal power flow
- 1,000 to 16,000 random scenarios



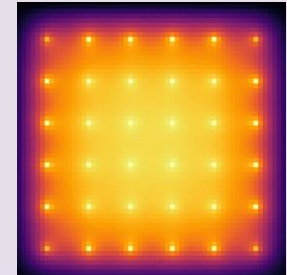
## Optimal Control

- Model predictive control of quadcopter
- Track trajectory setpoint and vary grid size



## PDE Optimal Control

- Control heating distribution
- 2D diffusion constraints



# NUMERICAL RESULTS (CPU W/ IPOPT)

- AD is **~20 times faster**
- Model build time is **1 – 2 orders-of-magnitude faster**

|                      | Optimal Quadrotor Control |       |        |        | 2D Heated Plate |       |        |        | Stochastic ACOPF |       |       |        |
|----------------------|---------------------------|-------|--------|--------|-----------------|-------|--------|--------|------------------|-------|-------|--------|
| Approach             | Build                     | AD    | Solve  | Total  | Build           | AD    | Solve  | Total  | Build            | AD    | Solve | Total  |
| JuMP.jl              | 42.56                     | 5.79  | 20.52  | 63.08  | 2.90            | 31.01 | 156.39 | 159.29 | 36.37            | 19.51 | 84.76 | 121.14 |
| MathOptSymbolicAD.jl | 43.80                     | 79.01 | 517.02 | 560.83 | 2.69            | 19.02 | 117.92 | 120.61 | 34.86            | 7.17  | 71.50 | 106.36 |
| AMPL                 | 51.83                     | 4.14  | 13.35  | 65.18  | 3.88            | 4.87  | 67.67  | 71.55  | 46.72            | 10.25 | 92.13 | 138.85 |
| ExaModelsMOI.jl      | 37.99                     | 0.47  | 9.60   | 47.59  | 3.06            | 2.33  | 115.51 | 118.57 | 52.54            | 1.40  | 84.15 | 136.69 |
| InfiniteExaModels.jl | 1.58                      | 0.37  | 9.04   | 10.62  | 0.06            | 1.43  | 91.16  | 91.22  | 0.30             | 0.80  | 81.17 | 81.47  |



- 1 `using InfiniteOpt, InfiniteExaModels, NLPModelsIpopt`
- 2 `model = InfiniteModel(ExaTranscriptionBackend(IpoptSolver))`

# NUMERICAL RESULTS (GPU W/ MADNLP)

- All AD and solve times are up to **~20 faster on GPU**
- InfiniteExaModels.jl builds models **orders-of-magnitude faster** than ExaModels

|                      | Optimal Quadrotor Control |      |       |       | 2D Heated Plate |       |       |       | Stochastic ACOPF |      |       |       |
|----------------------|---------------------------|------|-------|-------|-----------------|-------|-------|-------|------------------|------|-------|-------|
| Approach             | Build                     | AD   | Solve | Total | Build           | AD    | Solve | Total | Build            | AD   | Solve | Total |
| ExaModelsMOI.jl      | 37.34                     | 0.08 | 6.42  | 43.77 | 2.73            | 0.610 | 10.22 | 12.95 | 50.16            | 0.09 | 3.94  | 54.11 |
| InfiniteExaModels.jl | 1.31                      | 0.10 | 6.43  | 7.74  | 0.06            | 0.68  | 9.92  | 9.98  | 0.25             | 0.17 | 3.12  | 3.36  |

(63 on CPU)

(159 on CPU)

(121 on CPU)

```
1 using InfiniteOpt, InfiniteExaModels, MadNLPGPU, CUDA
2 transform_backend = ExaTranscriptionBackend(MadNLPSolver, backend = CUDABackend())
3 model = InfiniteModel(transform_backend)
```

# PAPER

- Learn more about recent advances



# TO THE TUTORIAL!

- Open the Part 3 Jupyter notebook in Colab

colab



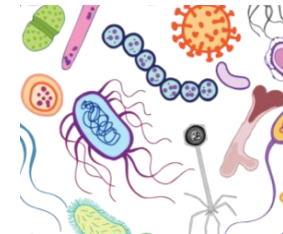
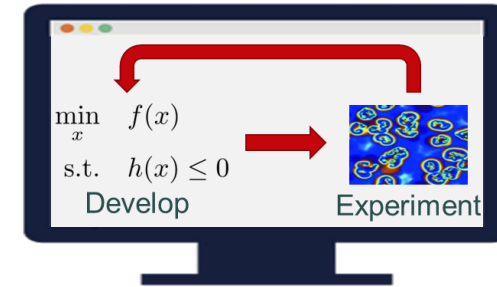
UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING



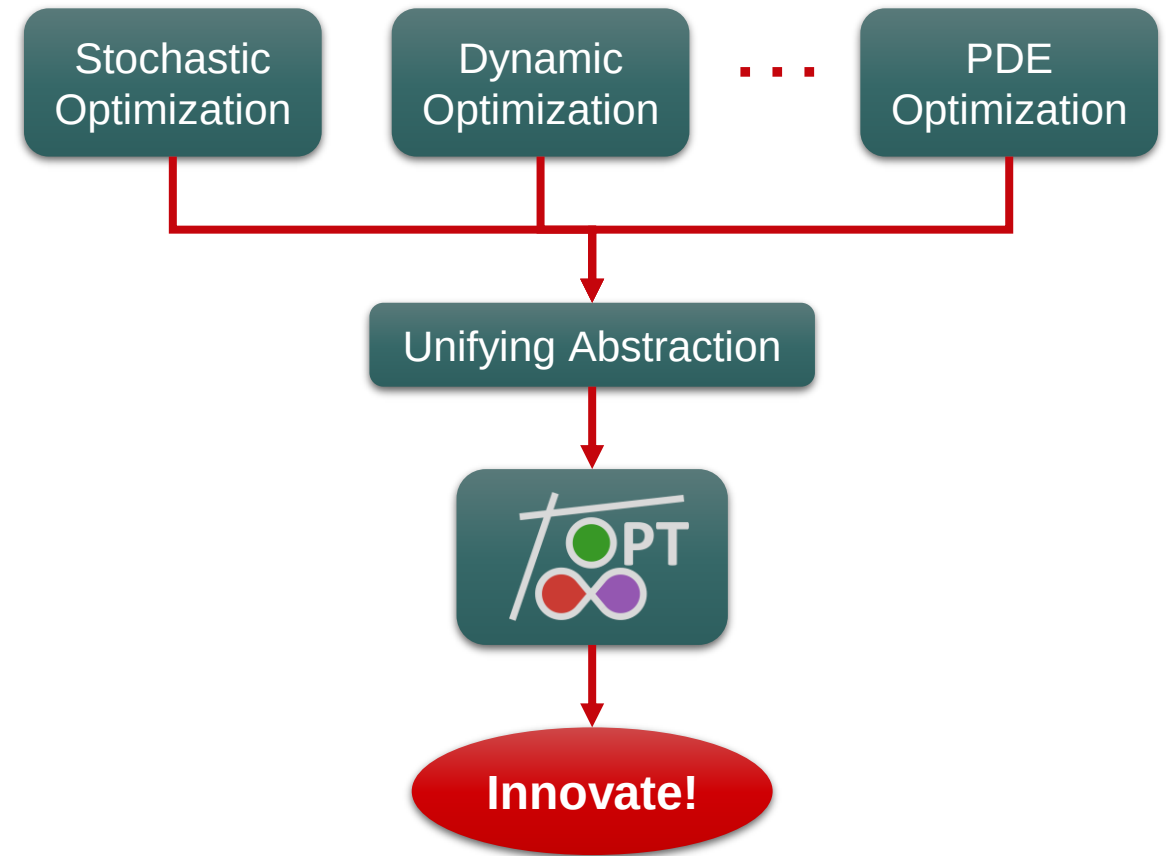
# WHEN SHOULD I USE INFINITEOPT.JL?

- Research
  - Sandbox InfiniteOpt problem approaches
  - Deploy your innovations → accessibility
  - Rapid comparisons
- Learning
  - High-level interface for novices
  - Features to interrogate and learn
- Acceleration
  - Solve problems faster on CPU/GPU via InfiniteExaModels.jl
- Application
  - Offline/online decision making (e.g., optimal control)
  - Any InfiniteOpt problem class



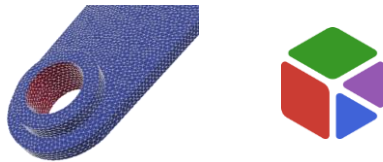
# WANT TO DEPLOY YOUR COOL TECHNIQUES?

- Almost all modeling aspects can be extended
  - **Infinite domain**
  - **Support generation**
  - **Measure** representation/evaluation
  - **Derivative** evaluation methods
  - Model **transformation**
  - More!
- Extensive **documentation, tutorials, & templates**



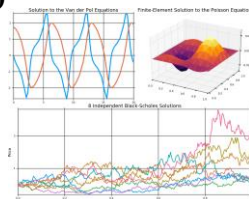
# FUTURE DEVELOPMENT: MODELING OBJECTS

- Non-rectangular domains
  - Leverage packages like Gridap.jl



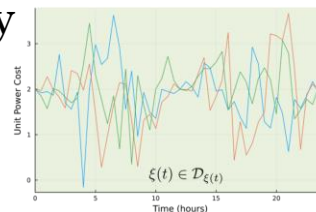
- Optimal Control Toolbox
  - Automatic initialization via DifferentialEquations.jl

- Simulation

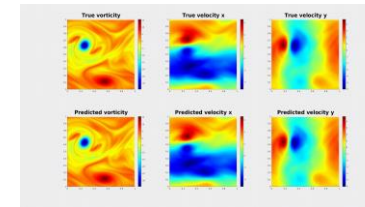


- Infinite Parameter Functions

- Embed random fields uncertainty

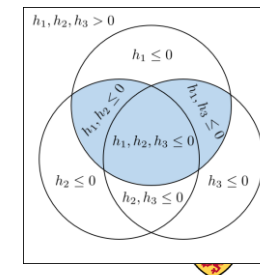


- Infinite-Dimensional Surrogates
  - Embed neural operator models



- Event Constraints

- Generalization of chance constraints
- Connection to generalized disjunctive programming



# HOW CAN I HELP?

- Get started with our tutorials: <https://pulsipher.github.io/InfiniteOpt.jl/stable/>
- Check out the contribution guide:  
[https://pulsipher.github.io/InfiniteOpt.jl/stable/develop/start\\_guide/](https://pulsipher.github.io/InfiniteOpt.jl/stable/develop/start_guide/)
- Reach out on the forum:  
<https://github.com/pulsipher/InfiniteOpt.jl/discussions>
- Give InfiniteOpt.jl a star!



# WHAT WERE THOSE RESOURCES, AGAIN?

- Julia
  - Julia learning: <https://julialang.org/learning/>
  - Documentation: <https://jump.dev/JuMP.jl/stable/>
  - Forum: <https://discourse.julialang.org/>
  - YouTube: <https://www.youtube.com/c/TheJuliaLanguage>
- JuMP.jl
  - Documentation: <https://jump.dev/JuMP.jl/stable/>
  - Forum: <https://discourse.julialang.org/c/domain/opt/13>
  - Free Textbook: <https://www.softcover.io/read/7b8eb7d0/juliabook2/introduction>
- InfiniteOpt.jl
  - Documentation: <https://pulsipher.github.io/InfiniteOpt.jl/stable/>
  - Forum: <https://github.com/pulsipher/InfiniteOpt.jl/discussions>



# THAT'S ALL!



UNIVERSITY OF  
**WATERLOO**

FACULTY OF  
ENGINEERING